

Implementation of a Homogeneous Self-Dual Interior-Point Method and Comparison of Predictor-Corrector Methods with Applications to Trajectory Optimization

Joe Koszut

May 19, 2026

1 Introduction

Many tasks in planning and control of dynamical systems can be cast as an optimization problem. However, solving these problems in a computationally efficient manner on the resource-constrained embedded platforms of many dynamical systems is a barrier to the broader adoption of optimization-based methods. Convex problems in particular have properties that facilitate strong guarantees with regard to traits such as convergence, feasibility, and global optimality, making them suitable for use on resource-constrained and safety-critical systems. Furthermore, a long line of work on sequential convex programming [1] has shown the importance of performant convex optimization methods even in the context of non-convex problems. All in all, it is reasonable to conclude that further research on performant and reliable IPMs for real-time trajectory optimization can provide meaningful impact. This project implements a homogeneous self-dual (HSD) interior-point method (IPM) and studies the impact of different predictor-corrector methods for it. The purpose is twofold: 1) enable the author to develop a stronger understanding of the details “under the hood” of a numerical solver and 2) study whether additional performance can be extracted from what is considered the present day state of the art in interior point methods for conic programming in the context of trajectory optimization.

2 Background

An optimal control or trajectory optimization problem aims to find the state and control trajectory for a dynamical system which will return the optimal performance with respect to a chosen metric, e.g., energy consumption or travel time, while satisfying various state and control constraints, e.g., distance from an obstacle or actuator torque limits. Once discretized, a trajectory optimization problem can be written in the following general form:

$$\min_{x_k, u_k} \quad \ell_N(x_N) + \sum_{k=0}^{N-1} \ell_k(x_k, u_k) \quad (1a)$$

$$\text{s.t.} \quad x_0 = x(0) \quad (1b)$$

$$x_{k+1} = f_k(x_k, u_k) \quad (1c)$$

$$g_k(x_k, u_k) \leq 0 \quad (1d)$$

$$g_N(x_N) \leq 0 \quad (1e)$$

$$h_k(x_k, u_k) = 0 \quad (1f)$$

$$h_N(x_N) = 0 \quad (1g)$$

for $k = 0, 1, \dots, N - 1$, with N being the horizon length, $x(0)$ the initial condition, x_k the state variables, u_k the control variables, l_k the stage cost, l_N the terminal cost, f_k the state dynamics, g_k the inequality constraints, and h_k the equality constraints.

Many trajectory optimization problems involve conic constraints on state or input variables. In particular, second-order cones (SOCs) can handle certain nonlinear constraints which arise in a variety of applications spanning spacecraft landing [2], robotics [3], and motorsports [4, 5]. A general conic program can be written in the following form:

$$\begin{aligned} \min_{x,s} \quad & c^\top x \\ \text{s.t.} \quad & Ax = b \\ & Gx + s = h \\ & s \in \mathcal{K} \end{aligned} \tag{P}$$

where $\mathcal{K}$ represents a closed and convex cone [6]. In most cases, $\mathcal{K}$ is made up of nonnegative orthant, second-order cone, and positive semidefinite cones. Each of these cones have a self-scaling property that will prove useful later in the development of efficient algorithms to solve (P). The Lagrangian of (P) can be expressed as

$$\mathcal{L}(x, s, y, z) = c^\top x + y^\top (Ax - b) + z^\top (Gx + s - h) \tag{2}$$

$$= x^\top (c + A^\top y + G^\top z) + s^\top z - b^\top y - h^\top z \tag{3}$$

The Lagrangian dual function can be expressed as

$$\mathcal{L}_D(y, z) = \inf_{x,s} \mathcal{L}(x, s, y, z) \tag{4}$$

$$= -b^\top y - h^\top z + \left(\inf_x x^\top (c + A^\top y + G^\top z) \right) + \left(\inf_s s^\top z \right) \tag{5}$$

Finding the infimum over x , we can see that it is finite if and only if

$$c + A^\top y + G^\top z = 0 \tag{6}$$

which gives a minimum of 0. Finding the infimum over s , we recall that if there exists a z_0 such that $s^\top z_0 < 0$, then by the nonnegative scaling invariance of a cone, we have $ts \in \mathcal{K}$ and hence $(ts)^\top z_0 \rightarrow -\infty$ as $t \rightarrow \infty$. Thus, the infimum is finite if and only if $s^\top z \geq 0 \forall s$. Notice that this is precisely the definition of the dual cone

$$\mathcal{K}^* = \{z \mid z^\top s \geq 0, \forall s \in \mathcal{K}\} \tag{7}$$

Therefore, in order for the infimum over s to be finite, we require $z \in \mathcal{K}^*$ which gives a minimum of 0. Thus, we can write the Lagrangian dual function as

$$\mathcal{L}_D(y, z) = -b^\top y - h^\top z \tag{8}$$

This allows us to write the Lagrangian dual problem as

$$\begin{aligned} \max_{y,z} \quad & -b^\top y - h^\top z \\ \text{s.t.} \quad & c + A^\top y + G^\top z = 0 \\ & z \in \mathcal{K}^* \end{aligned} \tag{D}$$

It will be assumed that when (P) and (D) are feasible, then (P) is strictly feasible, i.e., Slater's condition holds, and therefore strong duality holds, i.e., $p^* = d^*$.

The central path for (P) is defined uniquely for each $\mu > 0$ as the set of points which satisfy:

$$\begin{bmatrix} 0 \\ 0 \\ s \\ s \\ \kappa \end{bmatrix} = \begin{bmatrix} 0 & A^\top & G^\top & c \\ -A & 0 & 0 & b \\ -G & 0 & 0 & h \\ -c^\top & -b^\top & -h^\top & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ \tau \end{bmatrix} + \mu \begin{bmatrix} q_x \\ q_y \\ q_z \\ q_\tau \end{bmatrix}$$

$$(s, z) \succeq_K 0, \quad (\kappa, \tau) \geq 0, \quad s \circ z = \mu \mathbf{e}, \quad \tau \kappa = \mu \quad (9)$$

where x, s are the primal variables, y, z are the dual variables, and q_x, q_y, q_z , and q_τ are residuals that ensure any trivial initial guess is feasible. $s \circ z$ represents the Jordan product between s and z [7]. Search directions are found by linearizing (9) and applying Newton's method to take a combination of a predictor and a corrector step while iteratively decreasing the value of μ . The scalars τ and κ give homogeneity to (9) and guarantee that it is always feasible. If $\tau = 0$, this serves as a certificate of primal or dual infeasibility for $(\mathcal{P})$ or its dual, respectively. (9) is equivalent to the homogeneous self-dual embedding described in [8,9] and at the center of solvers such as CVXOPT [10] and ECOS [11]. The Karush-Kuhn-Tucker (KKT) conditions of $(\mathcal{P})$ are equivalent to (9) when the duality measure μ is zero.

Initial work from other researchers has shown the promise of customizing IPMs to exploit context-specific problem structure. Examples include using the approximate minimum degree algorithm (AMD) to reduce the cost of a frequently computed Cholesky factorization and taking advantage of fixed sparsity in the problem structure to reduce the computational effort associated with sparse linear algebra routines [12]. Although this work showed that a customized solver applied to a spacecraft landing problem could have a 2x improvement in runtime, this work did not study the impact of alternative scaling matrices or centering methods beyond the popular choices of Nesterov-Todd scaling [13,14] and Mehrotra predictor-corrector [15], respectively.

In Section 3, we derive the homogeneous self-dual embedding (HSDE) which is at the center of many modern solvers for conic programming. In Section 4, we describe the application of an interior-point method to the central path of the HSDE formulation. In Section 4.5, we introduce Gondzio's multiple centrality correctors. In Section 5, we introduce the conic formulation of the three degree-of-freedom (3-DoF) spacecraft landing problem. In Section 6, we compare the performance of the Mehrotra predictor-correct method against Gondzio's multiple centrality correctors. Finally, we conclude in Section 7 with a summary of the work and comments on future research avenues.

3 Derivation of Homogeneous Self-Dual Embedding (HSDE)

This section is inspired by Section 2 of the Clarabel paper [16]. It follows the same steps with the exceptions of using a linear objective as opposed to a quadratic one, and splits the $Ax + s = b$ constraint into the two $Ax = b$ and $Gx + s = h$ constraints as used in the CVXOPT paper [10]. The derivation in this section is presented generally for symmetric cones. However, the implementation and experiments which will follow are restricted to the nonnegative orthant and second-order cones for simplicity of initial implementation and analysis.

The Karush-Kuhn-Tucker (KKT) conditions for $(\mathcal{P})$ are

$$c + A^\top y + G^\top z = 0 \quad (\text{Stationarity}) \quad (10a)$$

$$s^\top z = 0 \quad (\text{Complementary slackness}) \quad (10b)$$

$$\left. \begin{array}{l} Ax = b \\ Gx + s = h \\ s \in \mathcal{K} \end{array} \right\} \quad (\text{Primal feasibility}) \quad (10c)$$

$$z \in \mathcal{K}^* \quad (\text{Dual feasibility}) \quad (10d)$$

The duality gap can be expressed as

$$p - d = c^\top x - (-b^\top y - h^\top z) \quad (11)$$

$$= c^\top x + b^\top y + h^\top z \quad (12)$$

We can perform substitutions using the KKT conditions to show that

$$s^\top z = (h - Gx)^\top z \quad (13)$$

$$= h^\top z - x^\top G^\top z \quad (14)$$

$$= h^\top z - x^\top (-c - A^\top y) \quad (15)$$

$$= h^\top z + c^\top x + (Ax)^\top y \quad (16)$$

$$= h^\top z + c^\top x + b^\top y \quad (17)$$

$$= p - d \quad (18)$$

In other words, the duality gap is exactly equal to the complementarity gap $s^\top z$. This is also sometimes referred to as the duality measure.

Combining problems $(\mathcal{P})$ and $(\mathcal{D})$ with an objective minimizing the duality gap, we can rewrite our problem as the following feasibility problem

$$\begin{aligned} \min_{x,s,y,z} \quad & s^\top z \\ \text{s.t.} \quad & c^\top x + b^\top y + h^\top z = 0 \\ & c + A^\top y + G^\top z = 0 \\ & Ax = b \\ & Gx + s = h \\ & (s, z) \in \mathcal{K} \times \mathcal{K}^* \end{aligned} \quad (19)$$

Note that the objective and first constraint both represent the duality gap. If we find a feasible point, it is guaranteed to be optimal for the original problem since the KKT conditions will be satisfied with zero duality gap. However, this problem is infeasible whenever $(\mathcal{P})$ or $(\mathcal{D})$ are infeasible. To formulate a problem which is always feasible, we can define the scalar iterates $(\hat{\kappa}, \hat{\tau}) \geq 0$ and apply the following change of variables

$$x \rightarrow \hat{x}/\hat{\tau}, \quad s \rightarrow \hat{s}/\hat{\tau}, \quad y \rightarrow \hat{y}/\hat{\tau}, \quad z \rightarrow \hat{z}/\hat{\tau} \quad (20)$$

Using this change of variables, we can rewrite the feasibility problem (19) as

$$\begin{aligned} \min_{\hat{x}, \hat{s}, \hat{y}, \hat{z}, \hat{\kappa}, \hat{\tau}} \quad & (\hat{s}/\hat{\tau})^\top (\hat{z}/\hat{\tau}) + \frac{\hat{\kappa}}{\hat{\tau}} \\ \text{s.t.} \quad & c^\top (\hat{x}/\hat{\tau}) + b^\top (\hat{y}/\hat{\tau}) + h^\top (\hat{z}/\hat{\tau}) = -\hat{\kappa} \\ & c + A^\top (\hat{y}/\hat{\tau}) + G^\top (\hat{z}/\hat{\tau}) = 0 \\ & A(\hat{x}/\hat{\tau}) = b \\ & G(\hat{x}/\hat{\tau}) + (\hat{s}/\hat{\tau}) = h \\ & (\hat{s}, \hat{z}, \hat{\kappa}, \hat{\tau}) \in \mathcal{K} \times \mathcal{K}^* \times \mathbb{R}_+ \times \mathbb{R}_+ \end{aligned} \quad (21)$$

Multiplying the objective and constraints by $\hat{\tau}^2$ and $\hat{\tau}$, respectively gives

$$\begin{aligned} \min_{\hat{x}, \hat{s}, \hat{y}, \hat{z}, \hat{\kappa}, \hat{\tau}} \quad & \hat{s}^\top \hat{z} + \hat{\kappa} \hat{\tau} \\ \text{s.t.} \quad & c^\top \hat{x} + b^\top \hat{y} + h^\top \hat{z} = -\hat{\kappa} \hat{\tau} \\ & c\hat{\tau} + A^\top \hat{y} + G^\top \hat{z} = 0 \\ & A\hat{x} = b\hat{\tau} \\ & G\hat{x} + \hat{s} = h\hat{\tau} \\ & (\hat{s}, \hat{z}, \hat{\kappa}, \hat{\tau}) \in \mathcal{K} \times \mathcal{K}^* \times \mathbb{R}_+ \times \mathbb{R}_+ \end{aligned} \quad (\mathcal{H})$$

which is known as the homogeneous self-dual embedding (HSDE). The formulation $(\mathcal{H})$ consists of two key properties

- **Self-duality:** Since $(\mathcal{H})$ encodes both the primal and dual programs, it is the dual of itself.
- **Homogeneity:** $(\mathcal{H})$ is now always feasible due to the homogeneity parameter $\hat{\tau}$. A trivial feasible point can be found by setting $\hat{\tau}$ and all other decision variables to 0. If a nontrivial feasible point is found where $\hat{\tau}$ and $\hat{\kappa}$ satisfy certain conditions discussed later, the optimal solution to the original problem can be extracted by dividing $\hat{x}$, $\hat{s}$, $\hat{y}$, and $\hat{z}$ by $\hat{\tau}$.

Writing out the linear constraints in $(\mathcal{H})$ in matrix-vector form gives

$$\begin{bmatrix} 0 \\ 0 \\ \hat{s} \\ \hat{\kappa} \end{bmatrix} = \begin{bmatrix} 0 & A^\top & G^\top & c \\ -A & 0 & 0 & b \\ -G & 0 & 0 & h \\ -c^\top & -b^\top & -h^\top & 0 \end{bmatrix} \begin{bmatrix} \hat{x} \\ \hat{y} \\ \hat{z} \\ \hat{\tau} \end{bmatrix} \quad (22)$$

Since the above matrix is skew-symmetric, it follows that left-multiplying the above equation by $[\hat{x}, \hat{y}, \hat{z}, \hat{\tau}]^\top$ gives $\hat{s}^\top \hat{z} + \hat{\kappa} \hat{\tau} = 0$ at all feasible points. Thus, we can equivalently express the objective function simply as 0 which implies that any feasible point is optimal.

3.1 Solution Extraction

To extract a solution for the original problem $(\mathcal{P})$, we consider the following cases¹:

- If $\hat{\tau} > 0$, $\hat{\kappa} = 0$, we can divide $\hat{x}$, $\hat{y}$, $\hat{z}$ by $\hat{\tau}$ to obtain a solution of the KKT conditions for $(\mathcal{P})$:

$$\begin{bmatrix} 0 \\ 0 \\ s \end{bmatrix} = \begin{bmatrix} 0 & A^\top & G^\top \\ -A & 0 & 0 \\ -G & 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \quad (s, z) \in \mathcal{K} \times \mathcal{K}^*, \quad s^\top z = 0 \quad (23)$$

- If $\hat{\tau} = 0$, $\hat{\kappa} > 0$, then $c^\top x + b^\top y + h^\top z < 0$. This implies we must have $c^\top x < 0$, $b^\top y + h^\top z < 0$, or both. If $b^\top y + h^\top z < 0$, this provides a certificate of primal infeasibility since if a valid primal solution existed satisfying $Ax = b$ and $Gx + s = h$ did exist, we would have

$$b^\top y + h^\top z = x^\top A^\top y + s^\top z + x^\top G^\top z = 0 \quad (24)$$

which is a contradiction. If $c^\top x < 0$, this provides a certificate of dual infeasibility since if a valid dual solution existed satisfying $c + A^\top y + G^\top z = 0$ did exist, we would have

$$c^\top x = -x^\top A^\top y - x^\top G^\top z = 0 \quad (25)$$

which is a contradiction.

- If $\hat{\tau} = \hat{\kappa} = 0$, this provides a certificate of facial reduction.

3.2 Extended Self-Dual Embedding

To start an interior-point method, we need an initial guess

$$(\hat{x}_0, \hat{y}_0, \hat{z}_0, \hat{\tau}_0, \hat{s}_0, \hat{\kappa}_0) \in \text{int}(\mathbb{R} \times \mathbb{R} \times \mathcal{K} \times \mathbb{R}_+ \times \mathcal{K}^* \times \mathbb{R}_+) \quad (26)$$

which also satisfies (22). This can sometimes be as hard as solving the original problem itself. To work around this, we can pick an arbitrary starting point that satisfies the strict inequalities, i.e.,

$$\hat{x}_0 = 0, \hat{y}_0 = 0, \hat{z}_0 = e, \hat{\tau}_0 = 1, \hat{s}_0 = e, \hat{\kappa}_0 = 1 \quad (27)$$

¹The first two conditions are taken directly from page 15 of the CVXOPT paper [10] and expanded slightly for clarity.

As this point is simply a trivial guess, it will almost surely not satisfy the linear system. Therefore, we introduce the vector q to represent the residual of our initial guess as

$$\begin{bmatrix} q_{\hat{x}} \\ q_{\hat{y}} \\ q_{\hat{z}} \\ q_{\hat{\tau}} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \hat{s}_0 \\ \hat{\kappa}_0 \end{bmatrix} - \begin{bmatrix} 0 & A^\top & G^\top & c \\ -A & 0 & 0 & b \\ -G & 0 & 0 & h \\ -c^\top & -b^\top & -h^\top & 0 \end{bmatrix} \begin{bmatrix} \hat{x}_0 \\ \hat{y}_0 \\ \hat{z}_0 \\ \hat{\tau}_0 \end{bmatrix} \quad (28)$$

If we embed the residual vector in our original linear system (22) and introduce the scalar variable θ , we obtain

$$\begin{bmatrix} 0 \\ 0 \\ \hat{s} \\ \hat{\kappa} \end{bmatrix} = \begin{bmatrix} 0 & A^\top & G^\top & c & q_{\hat{x}} \\ -A & 0 & 0 & b & q_{\hat{y}} \\ -G & 0 & 0 & h & q_{\hat{z}} \\ -c^\top & -b^\top & -h^\top & 0 & q_{\hat{\tau}} \end{bmatrix} \begin{bmatrix} \hat{x} \\ \hat{y} \\ \hat{z} \\ \hat{\tau} \\ \theta \end{bmatrix} \quad (29)$$

If we choose $(\hat{x}, \hat{y}, \hat{z}, \hat{\tau}) = (\hat{x}_0, \hat{y}_0, \hat{z}_0, \hat{\tau}_0)$ and set $\theta = 1$, the system is satisfied and we have a strictly feasible initial point. To maintain skew-symmetry, we add the following constraint

$$0 = q_{\hat{x}}^\top \hat{x} - q_{\hat{y}}^\top \hat{y} - q_{\hat{z}}^\top \hat{z} - q_{\hat{\tau}}^\top \hat{\tau} + (m+1) \quad (30)$$

m represents the degree of the cone that $\hat{s}$ and $\hat{z}$ live in, while 1 represents the degree of the cone that $\hat{\kappa}$ and $\hat{\tau}$ live in.

We can now represent the extended homogeneous self-dual embedding

$$\begin{aligned} & \min_{\hat{x}, \hat{s}, \hat{y}, \hat{z}, \hat{\kappa}, \hat{\tau}} (m+1)\theta \\ & \text{s.t.} \quad \begin{bmatrix} 0 \\ 0 \\ \hat{s} \\ \hat{\kappa} \\ 0 \end{bmatrix} = \begin{bmatrix} 0 & A^\top & G^\top & c & q_{\hat{x}} \\ -A & 0 & 0 & b & q_{\hat{y}} \\ -G & 0 & 0 & h & q_{\hat{z}} \\ -c^\top & -b^\top & -h^\top & 0 & q_{\hat{\tau}} \\ -q_{\hat{x}} & -q_{\hat{y}}^\top & -q_{\hat{z}}^\top & -q_{\hat{\tau}} & 0 \end{bmatrix} \begin{bmatrix} \hat{x} \\ \hat{y} \\ \hat{z} \\ \hat{\tau} \\ \theta \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ m+1 \end{bmatrix} \\ & (\hat{s}, \hat{z}, \hat{\kappa}, \hat{\tau}) \in \mathcal{K} \times \mathcal{K}^* \times \mathbb{R}_+ \times \mathbb{R}_+ \end{aligned} \quad (\mathcal{E})$$

Left-multiplying the linear constraint by $[\hat{x}, \hat{y}, \hat{z}, \hat{\tau}, \theta]^\top$ gives

$$\hat{s}^\top \hat{z} + \hat{\kappa} \hat{\tau} = (m+1)\theta \quad (31)$$

which allows us to express the objective function as $(m+1)\theta$. $(\mathcal{E})$ is referred to as the extended self-dual embedding (ESDE). Furthermore, (31) implies that the scale of initial infeasibility θ and the total complementarity gap $\hat{s}^\top \hat{z} + \hat{\kappa} \hat{\tau}$ are simultaneously driven to 0 as the problem is solved.

The optimality conditions for $(\mathcal{E})$ are

$$\begin{bmatrix} 0 \\ 0 \\ \hat{s} \\ \hat{\kappa} \\ 0 \end{bmatrix} = \begin{bmatrix} 0 & A^\top & G^\top & c & q_{\hat{x}} \\ -A & 0 & 0 & b & q_{\hat{y}} \\ -G & 0 & 0 & h & q_{\hat{z}} \\ -c^\top & -b^\top & -h^\top & 0 & q_{\hat{\tau}} \\ -q_{\hat{x}} & -q_{\hat{y}}^\top & -q_{\hat{z}}^\top & -q_{\hat{\tau}} & 0 \end{bmatrix} \begin{bmatrix} \hat{x} \\ \hat{y} \\ \hat{z} \\ \hat{\tau} \\ \theta \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ m+1 \end{bmatrix} \quad (32a)$$

$$(\hat{s}, \hat{z}, \hat{\kappa}, \hat{\tau}) \in \mathcal{K} \times \mathcal{K}^* \times \mathbb{R}_+ \times \mathbb{R}_+, \quad \hat{s}^\top \hat{z} + \hat{\kappa} \hat{\tau} = 0 \quad (32b)$$

Combined with (31), this implies that at optimum, $\theta = 0$ and the extended embedding reduces to the standard homogeneous embedding.

4 Interior Point Method

Although it can be simple to directly solve the linear system in (32a), it is much more difficult to directly satisfy the complementarity condition $\hat{s}^\top \hat{z} + \hat{\kappa} \hat{\tau} = 0$. Therefore, interior-point methods attempt to start within the interior of the feasible set and incrementally approach the complementarity condition while penalizing directions in which the iterates come in close proximity to the boundary of the feasible region. Although different options exist, the logarithmic barrier is used most commonly as it possesses various attractive properties. The logarithmic barrier function for $\mathcal{K}$ is

$$\phi(u) = \sum_{k=1}^K \phi_k(u_k), \quad \phi_k(u) = \begin{cases} -\sum_{j=1}^p \log(u_j) & \mathcal{K}_k = \mathbb{R}_+^p \\ -\frac{1}{2} \log(u_0^2 - u_1^\top u_1) & \mathcal{K}_k = \mathcal{Q}_p \\ -\log \det \text{mat}(u) & \mathcal{K}_k = \mathcal{S}_p \end{cases} \quad (33)$$

where $\mathcal{K}_k$ is the k -th cone in the Cartesian product of cones that make up $\mathcal{K}$. Note that $\phi(tu) = \phi(u) - m \log(t)$ for $t > 0$, where m is the *degree* of the cone $\mathcal{K}$ and is represented by

$$m = m_1 + \dots + m_K, \quad m_k = \begin{cases} p & \mathcal{K}_k = \mathbb{R}_+^p \\ 1 & \mathcal{K}_k = \mathcal{Q}_p \\ p & \mathcal{K}_k = \mathcal{S}_p \end{cases} \quad (34)$$

Note that some literature refers to m as the *rank* of $\mathcal{K}$. The gradients of ϕ and ϕ_k are $g(u) = \nabla \phi(u)$ and $g_k(u_k) = \nabla \phi_k(u_k)$ where

$$g_k(u_k) = \begin{cases} -\text{diag}(u_k)^{-1} \mathbf{1} & \mathcal{K}_k = \mathbb{R}_+^p \\ -(u_k^\top J u_k)^{-1} J u_k & \mathcal{K}_k = \mathcal{Q}_+^p \\ -\text{vec}(\text{mat}(u_k)^{-1}) & \mathcal{K}_k = \mathcal{S}_+^p \end{cases} \quad (35)$$

where

$$J = \begin{bmatrix} 1 & 0 \\ 0 & -I_{p-1} \end{bmatrix} \quad (36)$$

A property of $g(u)$ which will be used in the introduction of the central path is $u^\top g(u) = -m$ for $u \in \mathcal{K}$.

The Hessians of ϕ and ϕ_k are denoted $H(u) = \nabla^2 \phi(u)$ and $H_k(u_k) = \nabla^2 \phi_k(u_k)$ where

$$H_k(u_k) = \text{diag}(u_k)^{-2} \quad \mathcal{K}_k = \mathbb{R}_+^p \quad (37)$$

$$H_k(u_k) = \frac{1}{(u_k^\top J u_k)^2} (2J u_k u_k^\top J) - (u_k^\top J u_k) J \quad \mathcal{K}_k = \mathcal{Q}_+^p \quad (38)$$

$$H_k(u_k)v = -\text{vec}(\text{mat}(u_k)^{-1} \text{mat}(v) \text{mat}(u_k)^{-1}) \quad \mathcal{K}_k = \mathcal{S}_+^p \quad (39)$$

Definition 1. A useful property of the logarithmic barrier function is known as **self-scaling** which states that if $w \in \mathcal{K}$, then $H(w)u \in \mathcal{K}$ for all $u \in \mathcal{K}$.

A follow-on property from self-scaling is that $\phi(H(w)u) = \phi(u) - 2\phi(w)$.

4.1 Central Path

The central path of $(\mathcal{E})$ is defined as the solution of

$$\begin{bmatrix} 0 \\ 0 \\ \hat{s} \\ \hat{\kappa} \\ 0 \end{bmatrix} = \begin{bmatrix} 0 & A^\top & G^\top & c & q_{\hat{x}} \\ -A & 0 & 0 & b & q_{\hat{y}} \\ -G & 0 & 0 & h & q_{\hat{z}} \\ -c^\top & -b^\top & -h^\top & 0 & q_{\hat{\tau}} \\ -q_{\hat{x}} & -q_{\hat{y}} & -q_{\hat{z}} & -q_{\hat{\tau}} & 0 \end{bmatrix} \begin{bmatrix} \hat{x} \\ \hat{y} \\ \hat{z} \\ \hat{\tau} \\ \theta \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ m+1 \end{bmatrix} \quad (40a)$$

$$(\hat{s}, \hat{z}, \hat{\kappa}, \hat{\tau}) \in \mathcal{K} \times \mathcal{K}^* \times \mathbb{R}_+ \times \mathbb{R}_+, \quad \hat{z} = -\mu g(\hat{s}), \quad \hat{\tau} = \mu / \hat{\kappa} \quad (40b)$$

where $g(s)$ represents the gradient of the barrier function $\phi(\cdot)$. Using the property $u^\top g(u) = -m$ for $u \in \mathcal{K}$, it follows from (40b) that

$$\hat{s}^\top \hat{z} = \hat{s}^\top (-\mu g(s)) \quad (41)$$

$$= -\mu s^\top g(s) \quad (42)$$

$$= -\mu(-m) \quad (43)$$

$$= m\mu \quad (44)$$

Furthermore, we have $\hat{\kappa}\hat{\tau} = \mu$. Therefore, the total complementarity gap is

$$\hat{s}^\top \hat{z} + \hat{\kappa}\hat{\tau} = \mu(m+1) \quad (45)$$

which gives that $\mu = \frac{\hat{s}^\top \hat{z} + \hat{\kappa}\hat{\tau}}{m+1}$. If we take the inner product with $(\hat{x}, \hat{y}, \hat{z}, \hat{\tau}, \theta)$ on both sides of the equality (40a), we also get that $\theta = \mu$ on the central path. This allows us to express the central path (40) in the simpler form

$$\begin{bmatrix} 0 \\ 0 \\ \hat{s} \\ \hat{\kappa} \end{bmatrix} = \begin{bmatrix} 0 & A^\top & G^\top & c \\ -A & 0 & 0 & b \\ -G & 0 & 0 & h \\ -c^\top & -b^\top & -h^\top & 0 \end{bmatrix} \begin{bmatrix} \hat{x} \\ \hat{y} \\ \hat{z} \\ \hat{\tau} \end{bmatrix} + \mu \begin{bmatrix} q_{\hat{x}} \\ q_{\hat{y}} \\ q_{\hat{z}} \\ q_{\hat{\tau}} \end{bmatrix} \quad (46a)$$

$$(\hat{s}, \hat{z}, \hat{\kappa}, \hat{\tau}) \in \mathcal{K} \times \mathcal{K}^* \times \mathbb{R}_+ \times \mathbb{R}_+, \quad \hat{z} = -\mu g(\hat{s}), \quad \hat{\tau} = \mu/\hat{\kappa} \quad (46b)$$

The term $\hat{z} = -\mu g(\hat{s})$ is used as opposed to $\hat{s}^\top \hat{z} = \mu$ as the latter only enforces an average centrality, whereas the former enforces a cone-wise centrality.

If we take the inner product of each side of (46a) with $(0, 0, \hat{z}, \hat{\tau})$, we get

$$\hat{s}^\top \hat{z} + \hat{\kappa}\hat{\tau} = -\hat{z}^\top G\hat{x} + \hat{z}^\top h\hat{\tau} - \hat{\tau}c^\top \hat{x} - \hat{\tau}b^\top \hat{y} - \hat{\tau}h^\top \hat{z} + \mu\hat{z}^\top q_{\hat{z}} + \mu\hat{\tau}q_{\hat{\tau}} \quad (47)$$

$$= -\hat{x}^\top G^\top \hat{z} - \hat{\tau}c^\top \hat{x} - \hat{\tau}b^\top \hat{y} + \mu\hat{z}^\top q_{\hat{z}} + \mu\hat{\tau}q_{\hat{\tau}} \quad (48)$$

$$= -\hat{x}^\top (-A^\top \hat{y} - \hat{\tau}c - \mu q_{\hat{x}}) - \hat{\tau}c^\top \hat{x} - \hat{\tau}b^\top \hat{y} + \mu\hat{z}^\top q_{\hat{z}} + \mu\hat{\tau}q_{\hat{\tau}} \quad (49)$$

$$= \hat{x}^\top A^\top \hat{y} - \hat{\tau}b^\top \hat{y} + \mu\hat{x}^\top q_{\hat{x}} + \mu\hat{z}^\top q_{\hat{z}} + \mu\hat{\tau}q_{\hat{\tau}} \quad (50)$$

$$= (\mu q_{\hat{y}})^\top \hat{y} + \mu\hat{x}^\top q_{\hat{x}} + \mu\hat{z}^\top q_{\hat{z}} + \mu\hat{\tau}q_{\hat{\tau}} \quad (51)$$

$$= \mu(q_{\hat{x}}^\top \hat{x} + q_{\hat{y}}^\top \hat{y} + q_{\hat{z}}^\top \hat{z} + q_{\hat{\tau}}^\top \hat{\tau}) \quad (52)$$

which is exactly what the last equality in (40a) encodes. Hence, we do not need to explicitly write it in (46a). (46) will be used to parameterize the central path of $(\mathcal{H})$.

4.1.1 Euclidean Jordan Algebra

Furthermore, we can apply the concept of Euclidean Jordan algebra, a framework which generalizes the algebraic properties of symmetric matrices to symmetric cones [7, 17, 18]. Using the Jordan product, we can replace $\hat{z} = -\mu g(\hat{s})$ with $\hat{s} \circ \hat{z} = \mu \mathbf{e}$. The key benefit of this is that it allows us to develop numerical algorithms for symmetric cones which have the same high-level structure independent of the specific cone being used, e.g., Lorentz cone or PSD cone. For ease of implementation, the numerical algorithm which is implemented later will not handle the PSD cone. However, the overall structure that will be applied for the nonnegative orthant and Lorentz cones is also applicable to the PSD cone due to the use of Euclidean Jordan algebra.

Thus, the symmetric equivalent of the central path (46) can be expressed as

$$\begin{bmatrix} 0 \\ 0 \\ \hat{s} \\ \hat{\kappa} \end{bmatrix} = \begin{bmatrix} 0 & A^\top & G^\top & c \\ -A & 0 & 0 & b \\ -G & 0 & 0 & h \\ -c^\top & -b^\top & -h^\top & 0 \end{bmatrix} \begin{bmatrix} \hat{x} \\ \hat{y} \\ \hat{z} \\ \hat{\tau} \end{bmatrix} + \mu \begin{bmatrix} q_{\hat{x}} \\ q_{\hat{y}} \\ q_{\hat{z}} \\ q_{\hat{\tau}} \end{bmatrix} \quad (53a)$$

$$(\hat{s}, \hat{z}, \hat{\kappa}, \hat{\tau}) \in \mathcal{K} \times \mathcal{K}^* \times \mathbb{R}_+ \times \mathbb{R}_+, \quad \hat{s} \circ \hat{z} = \mu \mathbf{e}, \quad \hat{\kappa}\hat{\tau} = \mu \quad (53b)$$

where $\circ$ denotes the Jordan product which is defined by

$$u \circ v = \begin{cases} (u_1 v_1, \dots, u_p v_p) & \mathcal{K}_k = \mathbb{R}_+^p \\ (u^\top v, u_0 v_1 + v_0 u_1) & \mathcal{K}_k = \mathcal{Q}_p \\ \frac{1}{2} \text{vec}(\text{mat}(u)\text{mat}(v) + \text{mat}(v)\text{mat}(u)) & \mathcal{K}_k = \mathcal{S}_p \end{cases} \quad (54)$$

and $\mathbf{e}$ represents the identity vector $\mathbf{e} = (\mathbf{e}_1, \dots, \mathbf{e}_K)$ where

$$\mathbf{e}_k = \begin{cases} (1, 1, \dots, 1) & \mathcal{K}_k = \mathbb{R}^n \\ (1, 0, \dots, 0) & \mathcal{K}_k = \mathcal{Q}_p \\ \text{vec}(I_p) & \mathcal{K}_k = \mathcal{S}_p \end{cases} \quad (55)$$

which satisfies $\mathbf{e} \circ x = x \circ \mathbf{e} = x$, $\mathbf{e}^\top (s \circ z) = s^\top z$, and $\mathbf{e}^\top \mathbf{e} = m$.

4.2 Nesterov-Todd Scaling

At a high-level, interior-point methods are based on linearizing the central path equations, applying a Newton step, decreasing μ , and repeating. However, careful steps need to be taken in the implementation details such as to ensure that the algorithm does not become numerically ill-conditioned. In particular, as iterates approach optimality, we can get large discrepancies in magnitude between $\hat{s}$ and $\hat{z}$. This can make it difficult to take large primal-dual steps. Primal-dual scaling is used to alleviate this. The description which follows is based on [10].

A primal-dual scaling is a linear transformation of $\hat{s}$ and $\hat{z}$ that balances out the range in magnitude between $\hat{s}$ and $\hat{z}$ as $\mu \rightarrow 0$. In particular, a primal-dual scaling W gives

$$\tilde{s} = W^{-\top} s, \quad \tilde{z} = W z \quad (56)$$

which leaves the cone and central path invariant, i.e.,

$$s \in \mathcal{K} \iff \tilde{s} \in \mathcal{K}, \quad z \in \mathcal{K} \iff \tilde{z} \in \mathcal{K}, \quad s \circ z = \mu \mathbf{e} \iff \tilde{s} \circ \tilde{z} = \mu \mathbf{e} \quad (57)$$

Under the primal-dual scaling W , we can express the central path as

$$\begin{bmatrix} 0 \\ 0 \\ \hat{s} \\ \hat{\kappa} \end{bmatrix} = \begin{bmatrix} 0 & A^\top & G^\top & c \\ -A & 0 & 0 & b \\ -G & 0 & 0 & h \\ -c^\top & -b^\top & -h^\top & 0 \end{bmatrix} \begin{bmatrix} \hat{x} \\ \hat{y} \\ \hat{z} \\ \hat{\tau} \end{bmatrix} + \mu \begin{bmatrix} q_{\hat{x}} \\ q_{\hat{y}} \\ q_{\hat{z}} \\ q_{\hat{\tau}} \end{bmatrix} \quad (58a)$$

$$(\hat{s}, \hat{z}, \hat{\kappa}, \hat{\tau}) \in \mathcal{K} \times \mathcal{K}^* \times \mathbb{R}_+ \times \mathbb{R}_+, \quad (W^{-\top} \hat{s}) \circ (W \hat{z}) = \mu \mathbf{e}, \quad \hat{\kappa} \hat{\tau} = \mu \quad (58b)$$

The self-scaling property of the logarithmic barrier function provides a simple method for constructing a symmetric primal-dual scaling: the Hessian of the barrier at any strictly feasible point is a primal-dual scaling. However, other scaling matrices can also be used which do not need to be symmetric. Note that the primal-dual scaling changes at each iteration depending on the values of $\hat{s}$ and $\hat{z}$.

Nesterov-Todd (NT) scaling [13, 14] is a particular method for selecting W that is broadly adopted in conic programming. It is derived from the unique scaling point w satisfying

$$H(w) \hat{s} = \hat{z} \quad (59)$$

A Nesterov-Todd scaling point is obtained by factoring $H(w)$ as $H(w)^{-1} = W^\top W$, where W is a scaling matrix. A general expression for the scaling point is

$$w = H(\hat{s}^{-1/2}) \left(H(\hat{s}^{-1/2}) \hat{z} \right)^{-1/2} = H(\hat{z}^{1/2}) \left(H(\hat{z}^{-1/2}) \hat{s} \right)^{1/2} \quad (60)$$

Applying $H(w)^{-1} = W^\top W$ to (59) gives

$$H(w)\hat{s} = \hat{z} \quad (61)$$

$$\hat{s} = H(w)^{-1}\hat{z} \quad (62)$$

$$\hat{s} = W^\top W\hat{z} \quad (63)$$

$$W^{-\top}\hat{s} = W\hat{z} \quad (64)$$

We can define the scaled variable λ to represent the scaled conic primal and dual variables, i.e.,

$$\lambda = W^{-\top}\hat{s} = W\hat{z} \quad (65)$$

This results in the complementarity condition simplifying to

$$\lambda \circ \lambda = \mu \mathbf{e} \quad (66)$$

Although NT scaling has shown to be sufficiently performant in a broad range of problem cases, it is not clear whether other options such as Alizadeh-Haeberly-Overton (AHO) [19] can outperform it under specific problem classes. To focus on core algorithm implementation, we leave a comparison of NT scaling against AHO scaling, along with the naive case of no scaling at all to future work.

4.3 Path-Following Algorithm

Before discussing the algorithmic implementation a quick note on notation will be given to help provide clarity. Variables with a hat, e.g., $\hat{s}$, represent the current iterates of the algorithm. Variables with a tilde, e.g., $\tilde{s}$, represent variables that have had Nesterov-Todd scaling applied to them. Variables with an underscore a or c , e.g., $\hat{s}_a$, represent iterates for the affine or combined directions, respectively.

As previously stated, interior-point methods at a high-level are based on linearizing the central path equations, applying a Newton step, decreasing μ , and repeating. This can be referred to as a “path-following” algorithm as the iterates try to stay close to the central path across iterations. The algorithm which is implemented in this work is a replica of the 5-step path-following algorithm from Section 7 of the CVXOPT paper [10]. Below, we repeat the steps described in the original paper along with providing additional explanations to enhance the intuition behind each step.

1. The first step consists of generating a trivial feasible initial guess $(\hat{x}_0, \hat{y}_0, \hat{z}_0, \tau_0, \hat{\kappa}_0, \hat{s}_0)$ and computing the Nesterov-Todd scaling, residuals, and average complementarity gap. The residuals are computed as

$$\begin{bmatrix} r_{\hat{x}} \\ r_{\hat{y}} \\ r_{\hat{z}} \\ r_{\hat{\tau}} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \hat{s} \\ \hat{\kappa} \end{bmatrix} - \begin{bmatrix} 0 & A^\top & G^\top & c \\ -A & 0 & 0 & b \\ -G & 0 & 0 & h \\ -c^\top & -b^\top & -h^\top & 0 \end{bmatrix} \begin{bmatrix} \hat{x} \\ \hat{y} \\ \hat{z} \\ \hat{\tau} \end{bmatrix} \quad (67)$$

while the average complementarity gap is

$$\hat{\mu} = \frac{\hat{s}^\top \hat{z} + \hat{\kappa} \hat{\tau}}{m+1} = \frac{\lambda^\top \lambda + \hat{\kappa} \hat{\tau}}{m+1} \quad (68)$$

2. The second step computes the affine step direction which aims for $\tilde{s} \circ \tilde{z} = 0$. This is an aggressive target since it aims at zeroing the complementarity gap in one step. This affine step is computed by solving the following set of equations

$$\begin{bmatrix} 0 \\ 0 \\ \Delta \hat{s}_a \\ \Delta \hat{\kappa}_a \end{bmatrix} - \begin{bmatrix} 0 & A^\top & G^\top & c \\ -A & 0 & 0 & b \\ -G & 0 & 0 & h \\ -c^\top & -b^\top & -h^\top & 0 \end{bmatrix} \begin{bmatrix} \Delta \hat{x}_a \\ \Delta \hat{y}_a \\ \Delta \hat{z}_a \\ \Delta \hat{\tau}_a \end{bmatrix} = - \begin{bmatrix} r_{\hat{x}} \\ r_{\hat{y}} \\ r_{\hat{z}} \\ r_{\hat{\tau}} \end{bmatrix} \quad (69a)$$

$$\lambda \circ (W\Delta \hat{z}_a + W^{-\top}\Delta \hat{s}_a) = -\lambda \circ \lambda, \quad \hat{\kappa} \Delta \hat{\tau}_a + \hat{\tau} \Delta \hat{\kappa}_a = -\hat{\kappa} \hat{\tau} \quad (69b)$$

The first equation in (69b) is the linearization of the conic complementarity between $\hat{s}$ and $\hat{z}$ in scaled coordinates, i.e., $\tilde{s} \circ \tilde{z}$, around the current scaled iterate $(\tilde{s}, \tilde{z}) = (\lambda, \lambda)$. To prove this, we can write

$$(\lambda + \Delta \tilde{s}_a) \circ (\lambda + \Delta \tilde{z}_a) \approx \lambda \circ \lambda + \lambda \circ \Delta \tilde{z}_a + \lambda \circ \Delta \tilde{s}_a \quad (70)$$

Using the distributive property of the Jordan product, setting the linearization to 0, and writing out in terms of the unscaled iterates, we obtain

$$\lambda \circ \lambda + \lambda \circ (\Delta \tilde{z}_a + \Delta \tilde{s}_a) = 0 \quad (71)$$

$$\lambda \circ (\Delta \tilde{z}_a + \Delta \tilde{s}_a) = -\lambda \circ \lambda \quad (72)$$

$$\lambda \circ (W \Delta \hat{z}_a + W^{-\top} \Delta \hat{s}_a) = -\lambda \circ \lambda \quad (73)$$

which matches the first equation in (69b). A similar process can be followed to obtain the second equation which is the linearization of scalar complementarity between $\hat{\kappa}$ and $\hat{\tau}$.

One may wonder why we cannot take the full affine step which aims to drive the linear residuals and both conic and scalar complementarity gaps to 0. The reasoning is that the complementarity conditions are nonlinear. The linearization that is used to compute the affine step throws away higher-order terms. This could result in the gap potentially growing if a naive step is taken. Therefore, we compute a centering parameter and solve a second linear system to get a combined predictor-corrector direction which aims to simultaneously reduce the residuals while also reducing the total complementarity gap.

3. The third step computes the centering parameter used in the Mehrotra predictor-corrector method. Calculating the maximum affine step length gives

$$\alpha_a = \max\{\alpha \in [0, 1] \mid (\hat{s}, \hat{\kappa}, \hat{z}, \hat{\tau}) + \alpha(\Delta \hat{s}_a, \Delta \hat{\kappa}_a, \Delta \hat{z}_a, \Delta \hat{\tau}_a) \in \mathcal{K} \times \mathbb{R}_+ \times \mathcal{K}^* \times \mathbb{R}_+\} \quad (74)$$

$$= \max\{\alpha \in [0, 1] \mid (\lambda, \hat{\kappa}, \lambda, \hat{\tau}) + \alpha(W^{-\top} \Delta \hat{s}_a, \Delta \hat{\kappa}_a, W \Delta \hat{z}_a, \Delta \hat{\tau}_a) \in \mathcal{K} \times \mathbb{R}_+ \times \mathcal{K}^* \times \mathbb{R}_+\} \quad (75)$$

The total complementarity gap that would be induced by the full affine step is

$$\hat{\mu}_{\text{aff}} = \frac{(\hat{s} + \alpha_a \Delta \hat{s}_a)^\top (\hat{z} + \alpha_a \Delta \hat{z}_a) + (\hat{\kappa} + \alpha_a \Delta \hat{\kappa}_a)(\hat{\tau} + \alpha_a \Delta \hat{\tau}_a)}{m + 1} \quad (76)$$

As part of Mehrotra's predictor-corrector method, we calculate the centering parameter as

$$\sigma = \left(\frac{\hat{\mu}_{\text{aff}}}{\hat{\mu}} \right)^3 \quad (77)$$

$$\approx (1 - \alpha_a)^3 \quad (78)$$

where the second line comes from a heuristic implemented in the CVXOPT solver, as described on slide 19-28 of [20]. If α_a is large, the affine direction already stays well inside the cone so we can use a small σ to minimize the amount of centering. On the other hand, if α_a is small, it implies that the affine direction quickly leads the boundary of the feasible domain and it is not reducing complementarity effectively, so we choose a larger σ to provide more centering and stay closer to the central path. This allows us to stay away from the boundary of the feasible domain and take a larger step in the centered direction.

4. The fourth step computes the combined predictor-corrector direction to account for the linearization error of the affine step. The combined direction aims to simultaneously reduce the residuals while also maintaining a target level of complementarity. In particular, the Mehrotra predictor-corrector aims to find a corrector direction

$$(\Delta \hat{x}_m, \Delta \hat{y}_m, \Delta \hat{z}_m, \Delta \hat{\tau}_m, \Delta \hat{s}_m, \Delta \hat{\kappa}_m) \quad (79)$$

so that the combined direction

$$\Delta \hat{x}_c = \Delta \hat{x}_a + \Delta \hat{x}_m, \quad \Delta \hat{y}_c = \Delta \hat{y}_a + \Delta \hat{y}_m, \quad \Delta \hat{z}_c = \Delta \hat{z}_a + \Delta \hat{z}_m \quad (80)$$

$$\Delta \hat{\tau}_c = \Delta \hat{\tau}_a + \Delta \hat{\tau}_m, \quad \Delta \hat{s}_c = \Delta \hat{s}_a + \Delta \hat{s}_m, \quad \Delta \hat{\kappa}_c = \Delta \hat{\kappa}_a + \Delta \hat{\kappa}_m \quad (81)$$

satisfies

$$(\lambda + \Delta \tilde{s}_c) \circ (\lambda + \Delta \tilde{z}_c) \approx \sigma \hat{\mu} \mathbf{e}, \quad (\hat{\kappa} + \Delta \hat{\kappa}_c)(\hat{\tau} + \Delta \hat{\tau}_c) \approx \sigma \hat{\mu} \quad (82)$$

If we expand the left-hand side of the first equation above, we get

$$(\lambda + \Delta \tilde{s}_c) \circ (\lambda + \Delta \tilde{z}_c) = (\lambda + \Delta \tilde{s}_a + \Delta \tilde{s}_m) \circ (\lambda + \Delta \tilde{z}_a + \Delta \tilde{z}_m) \quad (83)$$

$$= \lambda \circ \lambda + \lambda \circ (\Delta \tilde{s}_a + \Delta \tilde{z}_a) + \lambda \circ (\Delta \tilde{s}_m + \Delta \tilde{z}_m) + \quad (84)$$

$$+ \Delta \tilde{s}_a \circ \Delta \tilde{z}_a + \Delta \tilde{s}_a \circ \Delta \tilde{z}_m + \Delta \tilde{s}_m \circ \Delta \tilde{z}_a + \Delta \tilde{s}_m \circ \Delta \tilde{z}_m \quad (85)$$

Mehrotra's method keeps the known quadratic term from the affine step but removes the unknown quadratic terms, giving the approximation

$$\lambda \circ \lambda + \lambda \circ (\Delta \tilde{s}_a + \Delta \tilde{z}_a) + \lambda \circ (\Delta \tilde{s}_m + \Delta \tilde{z}_m) + \Delta \tilde{s}_a \circ \Delta \tilde{z}_a = \sigma \hat{\mu} \mathbf{e} \quad (86)$$

Since we know from the affine step that $\lambda \circ (\Delta \tilde{z}_a + \Delta \tilde{s}_a) = -\lambda \circ \lambda$, we get

$$\lambda \circ (\Delta \tilde{s}_m + \Delta \tilde{z}_m) = -\Delta \tilde{s}_a \circ \Delta \tilde{z}_a + \sigma \hat{\mu} \mathbf{e} \quad (87)$$

Additionally, we have that

$$\lambda \circ (\Delta \tilde{s}_c + \Delta \tilde{z}_c) = \lambda \circ (\Delta \tilde{s}_a + \Delta \tilde{z}_a) + \lambda \circ (\Delta \tilde{s}_m + \Delta \tilde{z}_m) \quad (88)$$

$$= -\lambda \circ \lambda + \lambda \circ (\Delta \tilde{s}_m + \Delta \tilde{z}_m) \quad (89)$$

Substituting in (87) gives

$$\lambda \circ (\Delta \tilde{s}_c + \Delta \tilde{z}_c) = -\lambda \circ \lambda - \Delta \tilde{s}_a \circ \Delta \tilde{z}_a + \sigma \hat{\mu} \mathbf{e} \quad (90)$$

Writing out in terms of the unscaled iterates gives

$$\lambda \circ (W \Delta \hat{z}_c + W^{-\top} \Delta \hat{s}_c) = -\lambda \circ \lambda - (W^{-\top} \Delta \hat{s}_a) \circ (W \Delta \hat{z}_a) + \sigma \hat{\mu} \mathbf{e} \quad (91)$$

A similar process can be followed for $\hat{\kappa} \hat{\tau}$. Thus, the combined direction is computed by solving the following set of equations

$$\begin{bmatrix} 0 \\ 0 \\ \Delta \hat{s}_c \\ \Delta \hat{\kappa}_c \end{bmatrix} - \begin{bmatrix} 0 & A^\top & G^\top & c \\ -A & 0 & 0 & b \\ -G & 0 & 0 & h \\ -c^\top & -b^\top & -h^\top & 0 \end{bmatrix} \begin{bmatrix} \Delta \hat{x}_c \\ \Delta \hat{y}_c \\ \Delta \hat{z}_c \\ \Delta \hat{\tau}_c \end{bmatrix} = -(1 - \sigma) \begin{bmatrix} r_{\hat{x}} \\ r_{\hat{y}} \\ r_{\hat{z}} \\ r_{\hat{\tau}} \end{bmatrix} \quad (92a)$$

$$\lambda \circ (W \Delta \hat{z}_c + W^{-\top} \Delta \hat{s}_c) = -\lambda \circ \lambda - (W^{-\top} \Delta \hat{s}_a) \circ (W \Delta \hat{z}_a) + \sigma \hat{\mu} \mathbf{e} \quad (92b)$$

$$\hat{\kappa} \Delta \hat{\tau}_c + \hat{\tau} \Delta \hat{\kappa}_c = -\hat{\kappa} \hat{\tau} - \Delta \hat{\kappa}_a \Delta \hat{\tau}_a + \sigma \hat{\mu} \quad (92c)$$

Note that the same residuals computed in step 1 get used in both steps 2 and 4.

5. The fifth step updates the iterates using

$$(\hat{x}, \hat{y}, \hat{z}, \hat{\tau}, \hat{s}, \hat{\kappa}) = (\hat{x}, \hat{y}, \hat{z}, \hat{\tau}, \hat{s}, \hat{\kappa}) + \alpha (\Delta \hat{x}_c, \Delta \hat{y}_c, \Delta \hat{z}_c, \Delta \hat{\tau}_c, \Delta \hat{s}_c, \Delta \hat{\kappa}_c) \quad (93)$$

where

$$\alpha = \max \left\{ \alpha \in [0, 1] \mid (\lambda, \hat{\kappa}, \lambda, \hat{\tau}) + \frac{\alpha}{0.99} (W^{-\top} \Delta \hat{s}_c, \Delta \hat{\kappa}, W \Delta \hat{z}_c, \Delta \hat{\tau}) \right\} \quad (94)$$

The 0.99 scaling factor is used to allow us to take the largest possible step while still staying inside the strict interior of the feasible domain. Additionally, we compute the scaling matrix W for $\hat{s}$ and $\hat{z}$, along with the scaled point $\lambda = W^{-\top} \hat{s} = W \hat{z}$.

4.4 Linear System Simplification

The two linear systems in steps 2 and 4 differ only in the right-hand side and can be written in the form

$$\begin{bmatrix} 0 \\ 0 \\ \Delta\hat{s} \\ \Delta\hat{\kappa} \end{bmatrix} - \begin{bmatrix} 0 & A^\top & G^\top & c \\ -A & 0 & 0 & b \\ -G & 0 & 0 & h \\ -c^\top & -b^\top & -h^\top & 0 \end{bmatrix} \begin{bmatrix} \Delta\hat{x} \\ \Delta\hat{y} \\ \Delta\hat{z} \\ \Delta\hat{\tau} \end{bmatrix} = - \begin{bmatrix} d_{\hat{x}} \\ d_{\hat{y}} \\ d_{\hat{z}} \\ d_{\hat{\tau}} \end{bmatrix} \quad (95a)$$

$$\lambda \circ (W\Delta\hat{z} + W^{-\top}\Delta\hat{s}) = -d_{\hat{s}}, \quad \hat{\kappa}\Delta\hat{\tau}_a + \hat{\tau}\Delta\hat{\kappa}_a = -d_{\hat{\kappa}} \quad (95b)$$

The variables $\Delta\hat{s}$ and $\Delta\hat{z}$ can be eliminated from (95b) to give

$$\begin{bmatrix} 0 & A^\top & G^\top & c \\ -A & 0 & 0 & b \\ -G & 0 & W^\top W & h \\ -c^\top & -b^\top & -h^\top & 0 \end{bmatrix} \begin{bmatrix} \Delta\hat{x} \\ \Delta\hat{y} \\ \Delta\hat{z} \\ \Delta\hat{\tau} \end{bmatrix} = \begin{bmatrix} d_{\hat{x}} \\ d_{\hat{y}} \\ d_{\hat{z}} - W^\top(\lambda \diamond d_{\hat{s}}) \\ d_{\hat{\tau}} - d_{\hat{\kappa}}/\hat{\tau} \end{bmatrix} \quad (96a)$$

$$\Delta\hat{s} = -W^\top(\lambda \diamond d_{\hat{s}} + W\Delta\hat{z}), \quad \Delta\hat{\kappa}_a = -(d_{\hat{\kappa}} + \hat{\kappa}\Delta\hat{\tau}_a)/\hat{\tau} \quad (96b)$$

where $\diamond$ denotes the inverse operation. We can solve the 4x4 block system by solving two smaller KKT systems which use the following KKT matrix

$$\begin{bmatrix} 0 & A^\top & G^\top \\ -A & 0 & 0 \\ -G & 0 & W^\top W \end{bmatrix} \quad (97)$$

Section 7.4 in [10] can be referenced for additional implementation details regarding solving the linear system.

4.5 Gondzio's Multiple Centrality Correctors

In Mehrotra's predictor-corrector method, as described above, a single corrector step is used to compensate for the nonlinear complementarity terms neglected in the affine predictor. Gondzio's multiple centrality corrector (MCC) method extends this idea by allowing several additional correction steps within the same iteration. The goal is to improve centrality and, if possible, increase the admissible step length.

The MCC procedure can be summarized as follows:

1. Compute the Mehrotra combined direction $(\Delta\hat{x}_c, \Delta\hat{y}_c, \Delta\hat{z}_c, \Delta\hat{s}_c, \Delta\hat{\tau}_c, \Delta\hat{\kappa}_c)$
2. Evaluate the maximum raw step length α_c^{raw} that preserves feasibility, akin to the line searches performed in Section 4.3.
3. Using the current Nesterov-Todd scaling, form a trial point in scaled coordinates, as such

$$\tilde{s}^{\text{trial}} = \lambda + \alpha_c^{\text{raw}} W^{-\top} \Delta\hat{s}_c, \quad \tilde{z}^{\text{trial}} = \lambda + \alpha_c^{\text{raw}} W \Delta\hat{z}_c \quad (98)$$

4. Define the conewise trial complementarity in NT-scaled coordinates as

$$v_k^{\text{trial}} = \tilde{s}_k^{\text{trial}} \circ \tilde{z}_k^{\text{trial}} \quad (99)$$

where k corresponds to the k -th cone.

5. Define a target interval $[\beta\mu_{\text{target}}, \gamma\mu_{\text{target}}]$, where $\mu_{\text{target}} = \sigma\hat{\mu}\mathbf{e}_k$, σ is the centering parameter from step 3 of the path-following algorithm, and $0 \leq \beta \leq \gamma$. Values of $\beta = 0.5$ and $\gamma = 1.0$ were found to work well in limited numerical experiments.

6. Using the appropriate lower or upper bound of the target interval depending on the value of v^{trial} , calculate a correction vector $r_{\text{cor},\hat{s}\hat{z}} = v_{\hat{s}\hat{z}}^{\text{target}} - v_{\hat{s}\hat{z}}^{\text{trial}}$ corresponding to the complementarity error for $\hat{s}$ and $\hat{z}$. Likewise, calculate $r_{\text{cor},\hat{\kappa}\hat{\tau}}$ corresponding to the complementarity error for $\hat{\kappa}$ and $\hat{\tau}$.

For variables constrained to the nonnegative-orthant, the Jordan product is componentwise multiplication, so box correction is obtained by clipping each entry of $v_{\hat{\kappa}\hat{\tau}}^{\text{trial}}$ to the interval $[\beta\mu_{\text{target}}, \gamma\mu_{\text{target}}]$. For variables constrained to the Lorentz cone, the analogue of componentwise clipping is spectral clipping in the Euclidean Jordan algebra of the Lorentz cone. If $v_{\hat{s}\hat{z},k}^{\text{trial}} = (u_0, u_1)$, its Jordan eigenvalues are

$$\lambda_{\pm} = u_0 \pm \|u_1\|_2 \quad (100)$$

These eigenvalues are clipped to the admissible interval, giving λ^c , and the target element is reconstructed as

$$v_{\hat{s}\hat{z},k}^{\text{target}} = \left(\frac{\lambda_+^c + \lambda_-^c}{2}, \frac{\lambda_+^c - \lambda_-^c}{2} \frac{u_1}{\|u_1\|_2} \right) \quad (101)$$

with the vector (latter) part set to zero in the degenerate case $u_1 = 0$. The blockwise correction is then

$$r_{\text{cor},\hat{s}\hat{z},k} = v_{\hat{s}\hat{z},k}^{\text{target}} - v_{\hat{s}\hat{z},k}^{\text{trial}}, \quad (102)$$

7. Solve the following correction-only Newton system

$$\lambda \circ (W\Delta\hat{z}_{\text{cor}} + W^{-\top}\Delta\hat{s}_{\text{cor}}) = r_{\text{cor},\hat{s}\hat{z}} \quad (103)$$

$$\hat{\kappa}\Delta\hat{\tau}_{\text{cor}} + \hat{\tau}\Delta\hat{\kappa}_{\text{cor}} = r_{\text{cor},\hat{\kappa}\hat{\tau}} \quad (104)$$

8. Add the correction direction to the latest combined direction. Accept the correction only if the admissible step length improves sufficiently. A tolerance of $1e-3$ was used in the numerical experiments.
9. Repeat until no further improvement is obtained or a maximum number of corrector steps is reached. A maximum value of 3 was used in the numerical experiments. Including the additional Mehrotra corrector direction, this gives a maximum of 4 corrector steps in the experiments using Gondzio's MCC.

The practical motivation for Gondzio's method is that several inexpensive correction steps may enlarge the step length enough to reduce the total number of iterations. All correctors within an iteration use the same KKT matrix but a different right-hand side.

5 Trajectory Optimization Application

To benchmark the interior-point solver, we used a discretized three-degree-of-freedom (3-DoF) powered descent guidance (PDG) problem. The PDG problem involves computing a trajectory to guide a spacecraft from some arbitrary dynamic starting point in the atmosphere to a stationary point on the surface of a planet. This problem was chosen because it contains both linear and conic (Lorentz cones, in particular) constraints and is relevant to real-world engineering challenges given the renewed interest amongst government and industry to explore the Moon and Mars. Furthermore, it has a rich literature [2, 21–25]. Readers interested in the details of this problem and convexification methods for it are referred to [25]. As the practical problem formulation was not the focus of this work, the remainder of this section is a minimal description of the 3-DoF PDG problem.

5.1 Dynamics and discretization

Let $r(t) \in \mathbb{R}^3$, $v(t) \in \mathbb{R}^3$, $m(t) > 0$, and $T(t) \in \mathbb{R}^3$ denote position, velocity, mass, and thrust, respectively. The continuous-time dynamics are

$$\dot{r}(t) = v(t) \quad (105)$$

$$\dot{v}(t) = \frac{T(t)}{m(t)} + g_0 \quad (106)$$

$$\dot{m}(t) = -\alpha \|T(t)\|_2 \quad (107)$$

where $\alpha = \frac{1}{I_{sp}g_0}$ is used to convert thrust force to propellant mass flow rate. I_{sp} is the specific impulse of the engine and g_0 is a gravitational constant. To convexify the dynamics, the variables

$$u(t) = \frac{T(t)}{m(t)}, \quad z(t) = \log m(t) \quad (108)$$

are introduced, giving

$$\dot{r}(t) = v(t) \quad (109)$$

$$\dot{v}(t) = u(t) + g \quad (110)$$

$$\dot{z}(t) = -\alpha \|u(t)\|_2 \quad (111)$$

An auxiliary scalar σ_k is used to upper-bound the thrust magnitude

$$\|u_k\|_2 \leq \sigma_k \quad (112)$$

and it will also assist in convexifying the formulation. Here, σ_k denotes the thrust-magnitude slack variable and should not be confused with the centering parameter σ used in the predictor-corrector method. With this, the log-mass dynamics become linear in discrete time

$$z_{k+1} = z_k - \alpha \Delta t \sigma_k \quad (113)$$

The time horizon $[0, t_f]$ is divided into N uniform intervals of duration $\Delta t = \frac{t_f}{N}$. Assuming piecewise-constant control over each interval, the discretized dynamics are

$$r_{k+1} = r_k + \Delta t v_k + \frac{1}{2} \Delta t^2 (u_k + g) \quad (114)$$

$$v_{k+1} = v_k + \Delta t (u_k + g) \quad (115)$$

$$z_{k+1} = z_k - \alpha \Delta t \sigma_k \quad (116)$$

for $k = 0, \dots, N-1$. The initial conditions are

$$r_0 = r^{\text{init}} \quad (117)$$

$$v_0 = v^{\text{init}} \quad (118)$$

$$z_0 = \log m_0 \quad (119)$$

and the terminal conditions are

$$r_N = r_f, \quad v_N = v_f \quad (120)$$

5.2 State and Input Constraints

The original thrust bounds satisfy

$$\rho_1 \leq \|T_k\|_2 \leq \rho_2. \quad (121)$$

Since $T_k = e^{z_k} u_k$, these become

$$\rho_1 e^{-z_k} \leq \|u_k\|_2 \leq \rho_2 e^{-z_k}, \quad (122)$$

which are nonlinear in z_k . In the implemented convex model, these bounds are approximated affinely about a reference log-mass profile z_k^{ref}

$$\sigma_k \geq \mu_{1,k}^{\text{ref}} (1 - (z_k - z_k^{\text{ref}})) \quad \sigma_k \leq \mu_{2,k}^{\text{ref}} (1 - (z_k - z_k^{\text{ref}})) \quad (123)$$

where

$$\mu_{1,k}^{\text{ref}} = \rho_1 e^{-z_k^{\text{ref}}} \quad \mu_{2,k}^{\text{ref}} = \rho_2 e^{-z_k^{\text{ref}}} \quad (124)$$

The formulation also includes mass-envelope and dry-mass constraints

$$z_{\text{lb},k} \leq z_k \leq z_{\text{ub},k}, \quad z_k \geq \log m_{\text{dry}} \quad (125)$$

Lastly, we incorporate thrust tilt and glide-slope constraints

$$u_{z,k} \geq 0, \quad \|(u_{x,k}, u_{y,k})\|_2 \leq \tan(\theta_{\text{max}}) u_{z,k} \quad (126)$$

$$r_{z,k} \geq 0, \quad \|(r_{x,k}, r_{y,k})\|_2 \leq \tan(\gamma_{\text{gs}}) r_{z,k} \quad (127)$$

5.3 Objective

The objective is

$$\min \quad \Delta t \sum_{k=0}^{N-1} \sigma_k, \quad (128)$$

which corresponds to minimizing fuel usage, or equivalently maximizing terminal mass.

This powered descent problem provides conic program whose size can be varied through the discretization count N . In the experiments, the problem is modeled in CVXPY [26] and canonicalized into standard conic form. The resulting matrices and vectors (A, G, b, h, c) are then used as an input to the custom homogeneous self-dual interior-point solver. This provides a convenient and repeatable benchmark for evaluating the effect of predictor-corrector strategy on solver performance. Results are compared using the default solver in CVXPY to ensure no implementation errors exist.

6 Results and Discussion

6.1 Trajectory Optimization Solution

The optimized 3-DoF powered descent trajectory is shown in Figures 1 and 2. The solution satisfies the terminal landing conditions while respecting the thrust magnitude, thrust tilt, dry-mass, and glide-slope constraints. The position and velocity histories show a smooth descent to the landing site with all velocity components driven to zero at touchdown, indicating that the conic formulation captures the required terminal condition of the origin for both position and velocity. The ground distance versus altitude plot in Figure 2 confirms that the trajectory respects the glide-slope constraint. Altogether, these results suggest that the implemented conic transcription produces plausible results and is a suitable benchmark for comparing corrector strategies in the custom solver.

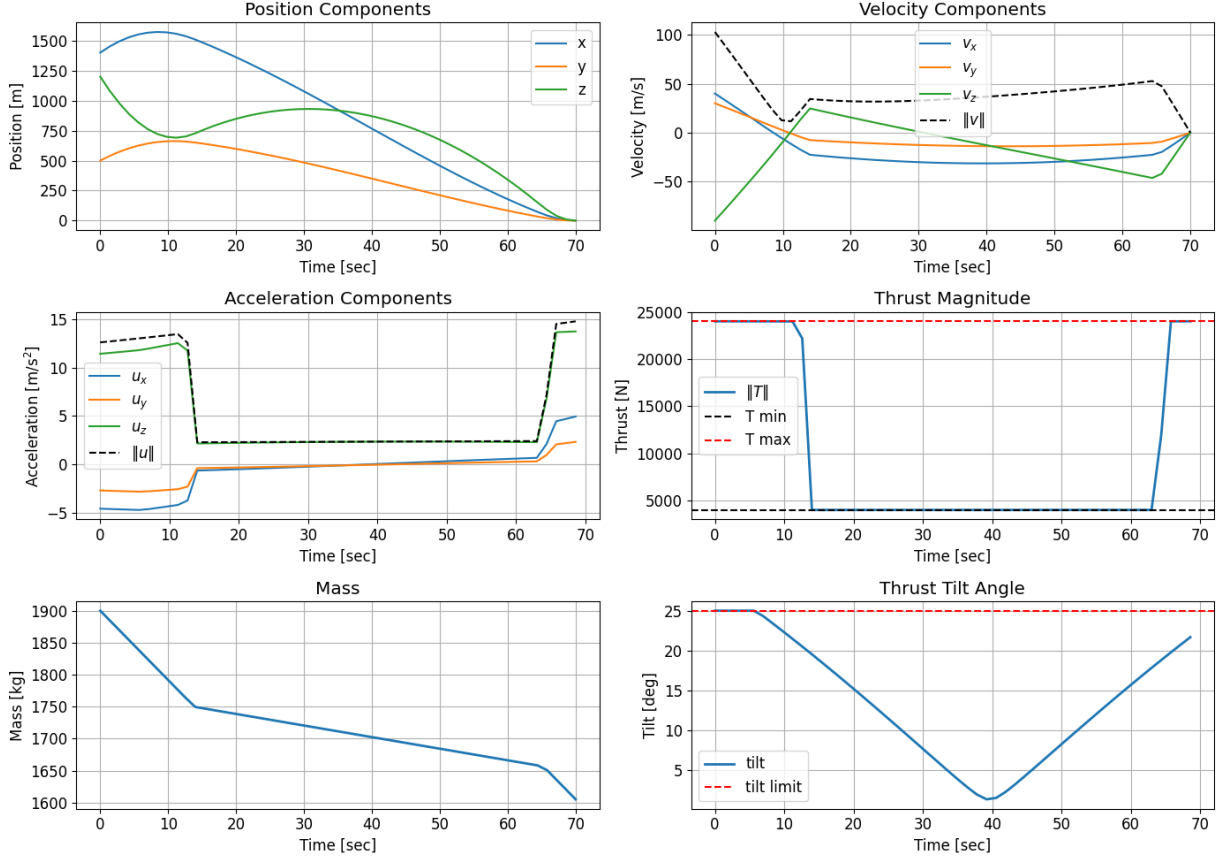


Figure 1: Summary of the computed 3-DoF landing trajectory showing satisfaction of the thrust magnitude constraints.

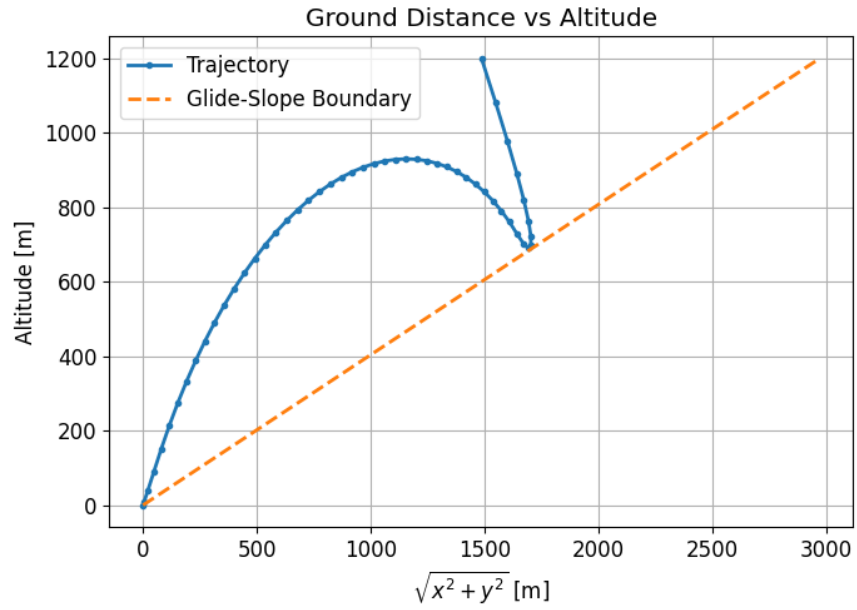


Figure 2: Spatial profile of the computed landing trajectory showing satisfaction of the glide-slope constraint.

6.2 Numerical Results

Figure 3 compares the convergence of the HSD interior-point solver using Mehrotra’s predictor-corrector method and Gondzio’s multiple centrality correctors on the same spacecraft landing problem instance. The most immediate observation is that Gondzio’s method reduces the complementarity gap

$$\hat{\mu} = \frac{\hat{s}^\top \hat{z} + \hat{\kappa} \hat{\tau}}{m+1}$$

substantially faster per iteration. In the tested case, Gondzio reaches the termination threshold in 13 iterations, whereas Mehrotra requires 20. This indicates that the additional centrality corrections are effective at improving the quality of the search direction and allowing more aggressive progress along the central path.

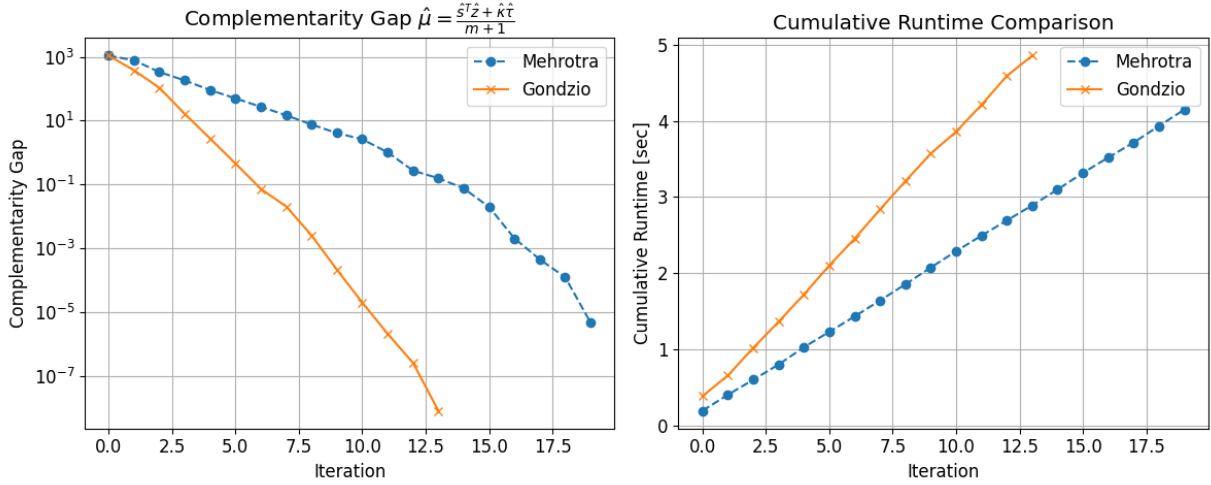


Figure 3: Comparison of solver performance between Mehrotra and Gondzio Corrector Methods.

Despite the reduction in iteration count, Gondzio was slower in cumulative runtime for this implementation. As shown in Figure 3, the Gondzio variant required approximately 4.8 seconds, while Mehrotra completed in about 4.1 seconds. This behavior is consistent with the additional work performed in each Gondzio iteration. The numerical results for the Gondzio method in Figure 4 show that, per iteration, the Gondzio method frequently used the maximum allowed three additional corrector steps on top of the standard Mehrotra corrector step. Each of these corrections required an additional linear solve, so the per-iteration cost increased enough to offset the reduction in total iteration count.

The numerical results for the Mehrotra method are shown in Figure 5. The main difference lies in step quality. In the Gondzio case, the combined step length α_{comb} remains close to one for most iterations, indicating that the additional corrector steps successfully recenter the iterates and enlarge the admissible fraction to the boundary. By contrast, the Mehrotra method typically takes smaller combined steps, especially in the middle of the solve. This explains why Gondzio makes faster progress in $\hat{\mu}$ per iteration.

7 Conclusion

This work implemented a minimal homogeneous self-dual IPM for linear cone programs over the nonnegative orthant and Lorentz cone. The Mehrotra predictor-corrector method popular in literature and practice was replaced with Gondzio’s multiple centrality correctors. A comparison of these variants was carried out on the spacecraft landing guidance problem posed as a conic program. Performance was quantified using time per iteration, total iteration count, and duality measure versus iteration. It was observed that Gondzio’s method is more effective in iteration count but not yet more efficient in total runtime.

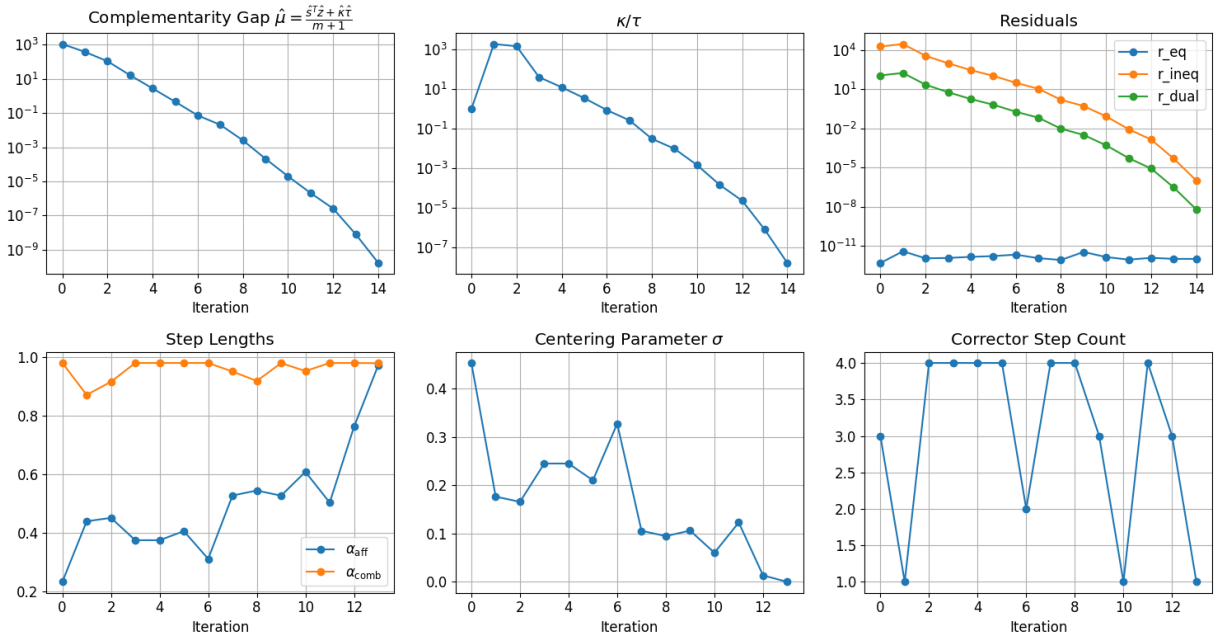


Figure 4: Details of solver convergence using the Gondzio corrector method.

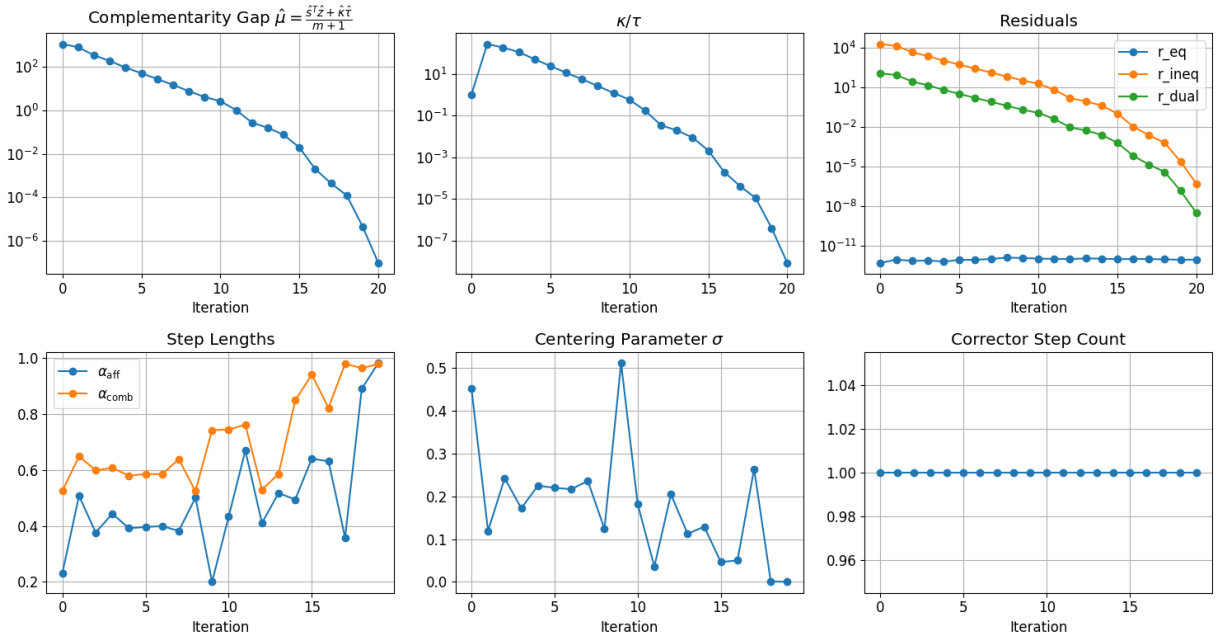


Figure 5: Details of solver convergence using the Mehrotra corrector method.

IPMs are broadly used for planning and control of dynamical systems but computational complexity can be a barrier for real-time implementation. The development of more performant general algorithms is an open area of research which can quickly encounter the “No Free Lunch” dilemma of optimization algorithms. Studying the details of algorithmic features on a specific problem class, however, offers the potential to make noteworthy performance improvements that extract the most from a given problem class. By improving the performance of IPMs, the challenge of applying real-time optimization-based methods to safety-critical autonomous systems can be reduced. The comparison of Mehrotra and Gondzio predictor-corrector strategies in this work hopefully provides useful insight into solver customization for compute-limited planning and control applications.

Completed steps include:

- a. Minimalistic implementation of the CVXOPT solver for conic linear programs.
- b. Minimalistic implementation of the powered descent guidance for spacecraft landing trajectory optimization problem.
- c. Comparison of performance between Mehrotra’s predictor-corrector and Gondzio’s multiple centrality corrector [27].

Desired future steps include:

- a. Comparing performance between Alizadeh-Haeberly-Overton (AHO) [19] and Nesterov-Todd [13, 14] scaling methods.
- b. Exploiting sparsity of the KKT system and identifying avenues for GPU parallelization.
- c. Expanding work to nonsymmetric cones.

References

- [1] A. G. Kamath, P. Elango, Y. Yu, S. Mceowen, G. M. Chari, J. M. Carson III, and B. Açıkmeşe, “Real-time sequential conic optimization for multi-phase rocket landing guidance,” *IFAC-PapersOnLine*, vol. 56, no. 2, pp. 3118–3125, 2023, 22nd IFAC World Congress. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2405896323018529>
- [2] B. Açıkmeşe, J. Casoliva, J. Carson, and L. Blackmore, “G-fold: A real-time implementable fuel optimal large divert guidance algorithm for planetary pinpoint landing,” *Concepts and Approaches for Mars Exploration*, vol. 1679, p. 4193, 2012.
- [3] M. S. Lobo, L. Vandenberghe, S. Boyd, and H. Lebert, “Applications of second-order cone programming,” *Linear Algebra and its Applications*, vol. 284, no. 1, pp. 193–228, 1998, international Linear Algebra Society (ILAS) Symposium on Fast Algorithms for Control, Signals and Image Processing. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0024379598100320>
- [4] T. Lipp and S. Boyd, “Minimum-time speed optimisation over a fixed path,” *International Journal of Control*, vol. 87, no. 6, pp. 1297–1311, 2014. [Online]. Available: <https://doi.org/10.1080/00207179.2013.875224>
- [5] S. Ebbesen, M. Salazar, P. Elbert, C. Bussi, and C. H. Onder, “Time-optimal control strategies for a hybrid electric race car,” *IEEE Transactions on Control Systems Technology*, vol. 26, no. 1, pp. 233–247, 2018.
- [6] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004.
- [7] L. Vandenberghe, “Symmetric cones,” Lecture notes for ECE236C: Optimization Methods for Large-Scale Systems, University of California, Los Angeles, 2016, accessed: 2026-05-11. [Online]. Available: <https://www.seas.ucla.edu/vandenbe/236C/lectures/symmetric.pdf>
- [8] Y. Ye, M. J. Todd, and S. Mizuno, “An $\mathcal{O}(\sqrt{n})$ -iteration homogeneous and self-dual linear programming algorithm,” *Mathematics of Operations Research*, vol. 19, no. 1, pp. 53–67, Feb. 1994. [Online]. Available: <http://dx.doi.org/10.1287/moor.19.1.53>

- [9] X. Xu, P.-F. Hung, and Y. Ye, “A simplified homogeneous and self-dual linear programming algorithm and its implementation,” *Annals of Operations Research*, vol. 62, no. 1, pp. 151–171, Dec 1996. [Online]. Available: <https://doi.org/10.1007/BF02206815>
- [10] L. Vandenberghe, “The cvxopt linear and quadratic cone program solvers,” Mar. 2010. [Online]. Available: <http://www.seas.ucla.edu/~vandenbe/publications/coneprog.pdf>
- [11] A. Domahidi, E. Chu, and S. Boyd, “ECOS: An SOCP solver for embedded systems,” in *European Control Conference (ECC)*, 2013, pp. 3071–3076.
- [12] D. Dueri, J. Zhang, and B. Açikmese, “Automated custom code generation for embedded, real-time second order cone programming,” *IFAC Proceedings Volumes*, vol. 47, no. 3, pp. 1605–1612, 2014, 19th IFAC World Congress. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1474667016418422>
- [13] Y. E. Nesterov and M. J. Todd, “Self-scaled barriers and interior-point methods for convex programming,” *Mathematics of Operations Research*, vol. 22, no. 1, pp. 1–42, 1997. [Online]. Available: <https://doi.org/10.1287/moor.22.1.1>
- [14] —, “Primal-dual interior-point methods for self-scaled cones,” *SIAM Journal on Optimization*, vol. 8, no. 2, pp. 324–364, 1998. [Online]. Available: <https://doi.org/10.1137/S1052623495290209>
- [15] S. Mehrotra, “On the implementation of a primal-dual interior point method,” *SIAM Journal on Optimization*, vol. 2, no. 4, pp. 575–601, 1992. [Online]. Available: <https://doi.org/10.1137/0802028>
- [16] P. J. Goulart and Y. Chen, “Clarabel: An interior-point solver for conic programs with quadratic objectives,” 2024.
- [17] J. Faraut and A. Korányi, *Analysis on Symmetric Cones*. Oxford University Press Oxford, Dec. 1994.
- [18] G.-Z. Gu, M. Zangiabadi, and A. Roos, “Full nesterov-todd step interior-point methods for symmetric optimization,” 2008. [Online]. Available: <https://api.semanticscholar.org/CorpusID:50576522>
- [19] F. Alizadeh, J.-P. A. Haeberly, and M. L. Overton, “Primal-dual interior-point methods for semidefinite programming: Convergence rates, stability and numerical results,” *SIAM Journal on Optimization*, vol. 8, no. 3, pp. 746–768, 1998. [Online]. Available: <https://doi.org/10.1137/S1052623496304700>
- [20] L. Vandenberghe, “Primal-dual interior-point methods,” Lecture notes for ECE236C: Optimization Methods for Large-Scale Systems, University of California, Los Angeles, 2016, accessed: 2026-05-11. [Online]. Available: <https://www.seas.ucla.edu/~vandenbe/236C/lectures/pd.pdf>
- [21] B. Açikmeşe and S. R. Ploen, “Convex programming approach to powered descent guidance for mars landing,” *Journal of Guidance, Control, and Dynamics*, vol. 30, no. 5, pp. 1353–1366, 2007. [Online]. Available: <https://doi.org/10.2514/1.27553>
- [22] B. Açikmeşe, D. Scharf, L. Blackmore, and A. Wolf, *Enhancements on the Convex Programming Based Powered Descent Guidance Algorithm for Mars Landing*. [Online]. Available: <https://arc.aiaa.org/doi/abs/10.2514/6.2008-6426>
- [23] L. Blackmore, B. Açikmeşe, and D. P. Scharf, “Minimum-landing-error powered-descent guidance for mars landing using convex optimization,” *Journal of Guidance, Control, and Dynamics*, vol. 33, no. 4, pp. 1161–1171, 2010. [Online]. Available: <https://doi.org/10.2514/1.47202>
- [24] B. Açikmeşe, J. M. Carson, and L. Blackmore, “Lossless convexification of nonconvex control bound and pointing constraints of the soft landing optimal control problem,” *IEEE Transactions on Control Systems Technology*, vol. 21, no. 6, pp. 2104–2113, 2013.
- [25] M. Szmuk, B. Açikmeşe, and A. W. Berning, *Successive Convexification for Fuel-Optimal Powered Landing with Aerodynamic Drag and Non-Convex Constraints*. [Online]. Available: <https://arc.aiaa.org/doi/abs/10.2514/6.2016-0378>
- [26] S. Diamond and S. Boyd, “CVXPY: A Python-embedded modeling language for convex optimization,” *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.
- [27] J. Gondzio, “Multiple centrality corrections in a primal-dual method for linear programming,” *Computational Optimization and Applications*, vol. 6, no. 2, p. 137–156, Sep. 1996. [Online]. Available: <http://dx.doi.org/10.1007/BF00249643>